Correlaci?n t?cnica del incidente P1-8841

Servicio: OrionOrders **Fecha de referencia:** 29 de septiembre de 2026 **Ventana de telemetría:** 09:35–10:41:30, según la fuente **Objetivo:** correlacionar aplicación, infraestructura, base de datos, servicios dependientes, CI/CD y cambios de la versión 4.18.0. **Estado:** análisis basado en evidencia entregada; no representa causa raíz formalmente confirmada ni ejecución de acciones en producción.

1. Resumen ejecutivo

La cadena técnica observada comienza con un release que reestructura el cliente HTTP de `customer-profile` y cambia el máximo documentado de conexiones de 200 a 20. Doce minutos después del rollout completo, a las 09:42, aumentan la latencia y la espera de adquisición del pool. A las 09:43:20 el log muestra 20/20 conexiones ocupadas y solicitudes pendientes. Desde entonces, crecen la espera local y las colas; más tarde aparecen 502/503, fallos de liveness y una disminución de pods listos.

En las solicitudes muestreadas, el tiempo esperando una conexión del cliente HTTP supera por mucho la duración registrada de la llamada a `customer-profile`. Las métricas agregadas de ese servicio se mantienen en el rango aproximado de 87–143 ms de p95 durante su ventana disponible. Esto apunta a contención local del pool, no demuestra que todas las llamadas downstream estuvieran sanas.

La alerta de DB no identifica la base usada por orders-api: los avisos son para `db-reporting-01`, mientras el datasource apunta a `orders-primary-01`. La instantánea de esta primaria a las 10:02 (118/300 conexiones, 44% CPU, cero lock waiters) no respalda saturación en ese momento.

La hipótesis de trabajo más consistente es que el límite reducido contribuyó a la saturación del pool; no está confirmada causalmente porque falta verificar el valor efectivo por pod, dimensionar la demanda y capacidad global del downstream, y comparar bajo carga controlada. Liveness, pods listos y HPA pueden haber amplificado el impacto. El log de aplicación termina a las 10:41:30 todavía degradado; no hay evidencia de recuperación ni conciliación de órdenes.

2. Arquitectura y rutas del flujo

2.1 Ruta síncrona de solicitud

```
Internet / cliente
   -> Balanceador de carga
       -> Kubernetes: orders-api (12 pods normalmente)
           -> PostgreSQL orders-primary (operaciones de orders-api)
           -> cliente HTTP -> customer-profile-service (HTTPS, síncrono)
           -> audit-service (asíncrono)
```

El HPA está configurado con mínimo 8, máximo 30 pods y objetivo de CPU 65%. Las sondas de readiness y liveness usan endpoints HTTP de `orders-api`; la documentación arquitectónica indica que liveness comparte el mismo servidor y grupo de hilos que el tráfico de negocio.

2.2 Interacciones y alcance de evidencia

Componente / enlace	Función documentada	Evidencia observada	Límite
Cliente → balanceador	Entrada de solicitudes y exposición del p95/5xx agregados.	p95 sube de 0.7 s a las 09:41 a 10.2 s a las 10:15; 5xx de 0.4% a 38%.	CSV termina a las 10:15; no hay agregado posterior.
Balanceador → pods `orders-api`	Distribución de tráfico a servicio en Kubernetes.	Pods listos pasan de 12 al inicio a 4 en la muestra de 10:08 y última muestra de 10:14; se registran fallos de liveness.	Muestras de un minuto/eventos puntuales; no hay detalle completo de endpoints por pod.

Componente / enlace	Función documentada	Evidencia observada	Límite
`orders-api` → pool HTTP	Espera por adquirir conexión antes de llamada a perfil de cliente.	`leased=20 max=20 pending=19` a las 09:43:20; los pendientes alcanzan 128 en las muestras tardías.	No hay series completas por pod ni detalle de adquisición/liberación del pool.
Pool HTTP → customer-profile	Llamada HTTPS síncrona usada en el camino de solicitud.	En varias solicitudes el pool wait es de segundos; duración registrada de la llamada suele ser ~89–168 ms. Métrica agregada p95 ~87–143 ms.	Métricas tienen distinta granularidad y alcance; no demuestran salud de todas las operaciones.
`orders-api` → PostgreSQL	Persistencia/consulta en `orders-primary-01`.	DBA a las 10:02 informa 118/300 conexiones, CPU 44%, cero lock waiters; spans DB muy inferiores al pool wait muestreado.	Instantánea puntual; no sustituye historia completa de la DB.
`orders-api` → audit-service	Auditoría asíncrona, según arquitectura.	No se aporta telemetría del servicio de auditoría en los archivos revisados.	No se puede cuantificar contribución, latencia ni errores de esa ruta.
Liveness/readiness → servidor de aplicación	Health checks HTTP, timeout 1 s; liveness falla tras 3 intentos.	Eventos de liveness fallan a las 09:45:12, 09:46:03, 09:48:41, 09:50:19 y 09:54:11.	El evento de 09:54:11 no tiene terminación asociada en el log de eventos.

3. Correlación por dominio y tiempo

Ventana	Aplicación y downstream	Infraestructura / entrada	Base de datos	Lectura correlacionada
09:21–09:32	No se reporta telemetría bajo carga; smoke de salud y orden de prueba pasa.	Build, pruebas, escaneo, Helm, deploy y rollout se marcan PASS.	No hay cambio SQL declarado en release.	Validaciones acreditan gates existentes, no comportamiento sostenido ante concurrencia.
09:35–09:41	Log app muestra 200 y pool wait 0 ms en muestras; APM pool wait 27–178 ms; downstream p95 87.4–137.8 ms.	Balanceador estable: p95 0.7 s, 5xx 0.4%; 12 pods listos en CSV.	Sin alerta específica de primary en el baseline entregado.	Referencia de operación observada antes de la degradación.
09:42	Pool wait aparece en log (159 ms) y APM (660 ms a 09:42:00; hasta 913 ms en muestras siguientes); tiempo de red 109–155 ms.	Balanceador registra p95 1.48 s y 5xx 4.94%; pods listos siguen en 12.	Span DB 62–72 ms en trazas cercanas.	Primer síntoma documentado coincide con crecimiento de espera local; p95 rompe SLO antes de la primera terminación de pod.
09:43–09:44	Pool alcanza 20/20 a 09:43:20; pendientes 19, luego 18–26; espera llega a 1778 ms a las 09:44:10; llamada downstream 90 ms.	Pods listos pasan a 11 en muestra de 09:44; balanceador a 09:44: p95 2.17 s, 5xx 6.57%.	Spans DB en trazas citadas permanecen en decenas de ms.	Saturación local observada precede a errores agregados >5% y a los eventos de liveness.
09:45–09:49	Solicitudes de 502/503 empiezan a partir de 09:46:50; muestras muestran pool wait ~2.2–2.7 s y downstream ~93–160 ms.	Liveness termina pods desde 09:45:12; pods listos llegan a 10; balanceador 09:49: p95 3.22 s, 5xx 10.66%; P1 declarado.	Alarma de conexiones 92% a las 09:47 se etiqueta como `db-reporting-01`.	Reinicios ocurren después de la espera; pueden reducir capacidad, pero no son el iniciador temporal demostrado. Alarma DB requiere atribución por host.
09:50–09:55	Espera de pool alcanza 3.5–4.4 s; pendientes 50–63; downstream 89–120 ms en ejemplos. A las 09:55 hay un status 200 con 4.63 s total.	Continúan fallos liveness; HPA fija desired size 18 a 09:52; CSV registra 9 pods listos; balanceador a 09:55: p95 5.24 s, 5xx 17.92%.	Alarma DB sube a 96% de conexiones y 81% CPU, aún sobre `db-reporting-01`.	Un 200 individual no es recuperación; el pool/cola y los SLI de extremo a extremo siguen degradados. HPA deseado no equivale a pods listos.

Ventana	Aplicación y downstream	Infraestructura / entrada	Base de datos	Lectura correlacionada
09:58–10:02	App: 4.995–6.133 s de pool wait en muestras; downstream 85–115 ms; 71–87 pendientes.	Pods listos varían 6–8; el contador de reinicios del CSV no es monotónico.	A 10:02 primary: 118/300 conexiones, CPU 44%, cero bloqueos.	Señales de aplicación siguen apuntando a espera local. La medición de la DB primary contradice su saturación en esa instantánea; la alerta original pertenece a otro servidor.
10:05–10:15	APM muestra pool wait 6.9–9 s; customer-profile agregado p95 120–143 ms.	Pods listos bajan a 4; balanceador llega a p95 10.2 s/38% 5xx.	No se aporta serie de la primary durante toda la ventana.	El deterioro es extremo y coincide con menos pods listos; no se identifica causalidad direccional entre esas señales.
10:20–10:41:30	Logs app registran espera de 9000 ms y 128 pendientes; hay respuestas 200 y 502/503 con duración cercana a 9.2–9.4 s.	No hay métricas agregadas LB ni pods posteriores a 10:15/10:14.	Sin nuevos datos de DB.	Evidencia de degradación persistente en las muestras de aplicación; no permite establecer la tasa agregada, la situación de infraestructura ni la hora de recuperación.

Las horas entre fuentes con y sin offset pueden no estar perfectamente alineadas. La tabla usa coincidencia temporal tal como se suministró y no interpola valores entre muestras.

4. Cambios de release y flujo CI/CD

4.1 Cambios declarados en 4.18.0

Cambio	Antes	Release 4.18.0	Relación técnica posible	Evidencia / límite
Máximo de conexiones HTTP a customer-profile	200	20	Un límite menor puede incrementar espera de adquisición si la concurrencia demandada supera la capacidad por instancia.	Release lo documenta; falta confirmar el valor cargado y la concurrencia por pod.
Connect timeout	1000 ms	1000 ms	Sin cambio documentado; no explica por sí solo el cambio de capacidad del pool.	Valores documentados en nota de release.
Read timeout	2000 ms	2000 ms	Sin cambio documentado.	Valores documentados en nota de release.
Cliente HTTP	Configuración anterior	Reestructurado	La nueva implementación puede haber cambiado adquisición, reutilización o liberación de conexiones.	No hay diff de código ni perfil detallado que permita atribuir un defecto de implementación.
Campo `salesChannel`	No indicado	Nuevo campo opcional	Cambio funcional adicional que debería cubrirse por contrato/pruebas; no hay señal que lo vincule con la saturación.	La release declara el cambio; sin evidencia de que afectara las trazas revisadas.
SQL	—	Sin cambios SQL declarados	Reduce la probabilidad de migración como desencadenante, pero no prueba que no haya efectos de consultas/carga.	Es declaración de release, no revisión del historial de DB.

4.2 Gates ejecutados y brechas

El pipeline registra build a las 09:21, unit tests (486) a las 09:23, análisis estático a las 09:25, escaneo de contenedor a las 09:26, `helm template` a las 09:28, deploy a las 09:29, rollout a las 09:30 y smoke tests hasta las 09:32, todos PASS. También indica expresamente:

- El esquema de Helm valida tipo y presencia del parámetro, no mínimo/máximo

operacionalmente aprobado para `CUSTOMER_PROFILE_MAX_CONNECTIONS`.

- No había prueba de carga similar a producción en el pipeline.
- Los smoke tests comprueban salud y una orden de prueba, no cola sostenida,

p95/p99 bajo concurrencia ni conexiones globales hacia customer-profile.

Por ello, «pipeline PASS» significa que pasaron los gates que existían; no demuestra que la configuración fuera adecuada para el patrón real de demanda.

5. Análisis de capacidad: por instancia y potencial global

La release documenta un máximo de 20 conexiones por cliente HTTP. Si ese límite se aplicara a cada pod, el máximo configurado agregado sería:

Pods	Máximo teórico con 20 conexiones/pod	Comparación con valor previo documentado de 200/pod
12 (normal)	240	2400
18 (tamaño deseado HPA a 09:52)	360	3600
30 (máximo HPA)	600	6000

Estos son productos aritméticos **condicionales**, no observaciones de conexiones realmente abiertas ni prueba de demanda. Presuponen que el valor es por instancia y que todos los pods alcanzan ese máximo. Los artefactos no especifican el presupuesto global aceptado por customer-profile, su límite de servidor, el número de pools reales ni el valor activo en cada pod.

Por tanto, escalar horizontalmente no garantiza una solución: si cada réplica crea su propio pool, el total de conexiones potenciales puede subir con el número de pods. Se debe validar el máximo agregado con el propietario del downstream antes de recomendar o aplicar un aumento de conexiones.

6. Hipótesis técnicas correlacionadas

H1 — Límite bajo del pool y saturación local

- **A favor:** cambio 200→20 documentado; 20/20 ocupadas; cola pendiente;

`clientPoolWaitMs` y spans de pool crecen; downstream medido es menor en las solicitudes citadas.

- **En contra / faltante:** configuración efectiva por pod y demanda concurrente

no disponibles; reestructuración de cliente también pudo introducir otro comportamiento; falta un experimento controlado.

- **Evaluación:** hipótesis principal; saturación observada con confianza alta,

causalidad específica con confianza moderada-alta, no formalmente confirmada.

H2 — Degradación originada por customer-profile

- **A favor:** es una dependencia síncrona y una degradación upstream puede

retener conexiones.

- **En contra / faltante:** spans de red y p95 agregado muestran tiempos bajos

comparados con la espera local en las muestras; hay huecos temporales y scopes diferentes.

- **Evaluación:** no es la explicación mejor respaldada; no puede descartarse

durante intervalos no medidos.

H3 — Saturación de PostgreSQL primary

- **A favor:** alertas DB fuertes durante el incidente.
- **En contra:** alertas etiquetadas `db-reporting-01`; datasource de orders-api

apunta a `orders-primary-01`; snapshot de primary a las 10:02 no muestra saturación; APM registra DB menor que la espera del pool.

- **Evaluación:** no respaldada para la primaria en la ventana observada; la

instantánea no permite una exclusión universal para todos los minutos.

H4 — Liveness, pérdida de pods y HPA

- **A favor:** fallos de liveness/terminaciones después del inicio; descenso de

pods listos; HPA cambia tamaño deseado; liveness usa servidor/grupo de hilos de negocio.

- **En contra / faltante:** no hay perfil de hilos, análisis por pod ni

contrafactual; no se demuestra que los pods deseados estuvieran listos a tiempo ni que el escalado aliviara el pool/downstream.

- **Evaluación:** mecanismo de amplificación plausible; contribución

cuantitativa no determinada. No es causa inicial demostrada.

7. Conclusiones por nivel de confianza

Conclusión	Confianza	Justificación
El pool HTTP estuvo saturado en varias muestras y hubo cola local.	Alta	Logs indican 20/20 ocupadas y pendientes desde 09:43:20, continuando en muestras posteriores.
La espera de adquisición explica gran parte de la latencia de solicitudes muestreadas.	Alta para ejemplos; moderada-alta como patrón	Tiempos de pool de segundos frente a llamadas downstream de decenas/pocos cientos de ms; APM corrobora en muestras.
La reducción de 200 a 20 causó la saturación.	Moderada-alta, no confirmada	Cambio documentado y secuencia consistentes, pero falta valor efectivo/demanda/prueba aislada.
La base de datos primary fue la causa del incidente.	Baja / no respaldada	La alerta pertenece a otro host; snapshot primary no indica saturación a las 10:02.
Fallos de liveness y menor cantidad de pods amplificaron la incidencia.	Moderada	Cronología y acoplamiento arquitectónico son compatibles, pero no cuantificados.
El escalado HPA resolvió la saturación.	No demostrado	Se conoce el tamaño deseado, pero no recuperación de pods ni alivio de pool.
El incidente terminó y todas las órdenes quedaron íntegras.	No verificable	Último log sigue degradado; no hay telemetría posterior ni conciliación de órdenes.

8. Evidencia adicional necesaria

1. Configuración efectiva (ConfigMap, Helm y variables materializadas) por pod y hora, junto con versión exacta del contenedor durante el rollout.
1. Demanda de solicitudes concurrentes por pod y métricas completas del pool: ocupadas/libres, pending, acquire timeout, duración de lease y errores.
1. Límite de conexiones y capacidad global acordados con el equipo de customer-profile, incluidos otros consumidores del servicio.
1. Trazas correlacionadas por request/pod que distingan pool wait, tiempo de red, DB y auditoría asíncrona.
1. Historial de pods listos, endpoints del balanceador, HPA y eventos; resolver el contador `cumulative_restarts` no monotónico.
1. Serie DB histórica por host, base/datasource, conexiones, CPU, bloqueos y tiempos de consulta durante el incidente.
1. Telemetría agregada posterior a 10:15 para balanceador/downstream y posterior a 10:41:30 para orders-api.
1. Prueba controlada y aprobada en ambiente seguro que modifique una sola variable y mida carga, latencia, cola y presupuesto global.
1. Conciliación de órdenes por identificadores/estados a través de un procedimiento autorizado de solo lectura. No modificar directamente datos.

9. Implicaciones para recuperación y prevención

- Antes de ajustar conexiones, verificar capacidad global de customer-profile; no asumir que 200 por pod es seguro.
- Comparar una corrección acotada de ConfigMap/Helm con rollback a 4.17.6, documentando alcance, aprobador, criterios de detención y reversión.
- No escalar pods como acción aislada sin calcular el total de conexiones potenciales y comprobar que el downstream puede atenderlas.
- Añadir métricas y alertas separadas para pool wait/pending, llamadas downstream, p95/p99 y estados HTTP; etiquetar las alertas DB con el host y datasource correctos.
- Hacer que CI valide rango aprobado y conexiones máximas agregadas; agregar carga representativa y evaluar liveness bajo presión antes del despliegue.
- No declarar recuperación por `/health` o por respuestas 200 aisladas: validar p95 < 1.2 s, p99 < 2.5 s, 5xx < 5% sostenidos según el criterio operativo aprobado y verificar integridad de órdenes por separado.

10. Fuentes

- `ORIONORDERS/00_DOCUMENTACION_GENERAL/01_ARQUITECTURA(3).md`
- `ORIONORDERS/01_SOPORTE_INCIDENTE/02_P1-8841(2).txt`
- `ORIONORDERS/01_SOPORTE_INCIDENTE/15_current_slo(2).md`
- `ORIONORDERS/01_SOPORTE_INCIDENTE/16_chat_crisis(2).txt`
- `ORIONORDERS/02_APLICACION_DESARROLLO/05_orders-api(2).log`
- `ORIONORDERS/02_APLICACION_DESARROLLO/08_customer_profile_metrics(2).csv`
- `ORIONORDERS/02_APLICACION_DESARROLLO/13_apm_trace_breakdown(2).csv`
- `ORIONORDERS/03_INFRAESTRUCTURA/06_k8s_pod_metrics(2).csv`
- `ORIONORDERS/03_INFRAESTRUCTURA/07_k8s_events(2).log`
- `ORIONORDERS/03_INFRAESTRUCTURA/10_load_balancer_metrics(2).csv`
- `ORIONORDERS/03_INFRAESTRUCTURA/11_health_probe_config(2).yaml`
- `ORIONORDERS/04_BASE_DE_DATOS/09_db_alerts(2).txt`
- `ORIONORDERS/05_DEVOPS_CICD_RELEASE/03_RELEASE_4.18.0(2).md`
- `ORIONORDERS/05_DEVOPS_CICD_RELEASE/04_pipeline_4.18.0(3).log`
- `ORIONORDERS/06_CONFIGURACION/12_http_client_config(2).yaml`
- `ORIONORDERS/07_SEGURIDAD/17_security_notes(2).md`
